Visão geral do trabalho

Problemas, entregáveis e convenções adotadas

Instituição: Universidade de Fortaleza (Unifor)
Curso: Ciência da Computação
Integrante: João Victor dos Santos Carvalho — matrícula 2510482
Modalidade: trabalho individual

Objetivo

Implementar dois problemas clássicos de divisão e conquista, comparar as soluções com força bruta e verificar se os tempos medidos se comportam de acordo com as complexidades previstas. O trabalho não utiliza algoritmos de ordenação como problema principal.

Entregáveis organizados

Código: implementações em algoritmos.py e interface em main.py.
Testes: casos_teste.json com entradas e saídas esperadas, mais test_algoritmos.py.
Experimentos: benchmark.py, medicoes_brutas.csv, resumo_medicoes.csv, gráficos e configuração da máquina.
Documentação: README.md e este relatório PDF.

Conceitos gerais

Força bruta enumera possibilidades sem explorar a estrutura recursiva. Divisão e conquista divide a instância em duas partes, resolve cada uma e combina os resultados. Foram implementadas três opções para subarranjo e duas para pontos.

Convenções de saída

Os índices começam em 0. Em subarranjos, início e fim são inclusivos e um resultado não pode ser vazio. Em empate de soma, escolhemos o menor índice de início e, depois, o menor índice de fim. Para pontos, a distância retornada é euclidiana.

Síntese das implementações

Problema	Método	Complexidade
Subarranjo	Força bruta	$\Theta(n^2)$
Subarranjo	Divisão e conquista	$\Theta(n \log n)$
Subarranjo	Linear (desafio)	$\Theta(n)$
Pontos	Força bruta	$\Theta(n^2)$
Pontos	Divisão e conquista	$\Theta(n \log n)$

Etapa 1 — Subarranjo máximo

Soma máxima de um intervalo contínuo

Definição do problema

Dado um vetor de n inteiros positivos ou negativos, encontrar o intervalo contínuo cuja soma é maior. Exemplo: no vetor [-2, 1, -3, 4, -1, 2, 1, -5, 4], o resultado é soma 6, índices [3, 6], correspondente a [4, -1, 2, 1].

Força bruta — O(n²)

Para cada posição inicial i, percorremos as posições finais j ≥ i e atualizamos incrementalmente a soma do intervalo. São n(n+1)/2 intervalos, cada atualização é O(1) e, portanto, o tempo total é Θ(n²). Essa versão serve de referência de correção.

Divisão e conquista — O(n log n)

Divide-se o vetor ao meio. A melhor resposta ou está totalmente à esquerda, totalmente à direita, ou cruza o meio. A parte cruzada é calculada por uma varredura de sufixos à esquerda e outra de prefixos à direita, ambas em tempo linear. Seleciona-se a melhor das três soluções.

Recorrência: $T(n) = 2T(n/2) + \Theta(n)$, com $T(1) = \Theta(1)$.
Solução: pelo Teorema Mestre, $a = 2$, $b = 2$, $f(n) = \Theta(n) = \Theta(n^{\log_2 2})$; logo $T(n) = \Theta(n \log n)$. Espaço auxiliar: O(log n) da pilha recursiva.

Questão-desafio — O(n)

Como o critério de avaliação menciona uma versão O(n) sem detalhar a questão no corpo do enunciado, acrescentamos a alternativa linear de Kadane. Para cada posição, a melhor soma terminando ali é o maior valor entre iniciar um novo intervalo e estender o anterior. Há uma atualização O(1) por elemento: Θ(n).

Casos de borda

Em vetor apenas negativo, retorna o maior elemento, não zero; com um único elemento, retorna esse elemento; em empates, aplica a convenção de índices do projeto. Os três métodos implementam o mesmo critério.

Passos principais da combinação

1. Resolver recursivamente os intervalos [esq, meio] e [meio+1, dir].
2. Percorrer os sufixos que terminam em meio e registrar o melhor.
3. Percorrer os prefixos que começam em meio+1 e registrar o melhor.
4. Comparar esquerda, direita e cruzado, mantendo início e fim.

Etapa 2 — Pontos mais próximos

Combinação geométrica e análise de recorrência

Força bruta — O(n²)

Para cada par distinto de coordenadas (x, y), calcula-se a distância euclidiana $d = \sqrt{[(x_1-x_2)^2 + (y_1-y_2)^2]}$. A implementação compara distâncias ao quadrado para evitar raízes quadradas repetidas e aplica a raiz somente na saída. Existem $n(n-1)/2$ pares: $\Theta(n^2)$.

Divisão e conquista — O(n log n)

Os pontos são ordenados inicialmente por x e por y (biblioteca padrão). A lista ordenada em x é dividida na mediana vertical; a lista em y é repartida preservando sua ordenação. Cada metade retorna sua menor distância. Se δ é a menor distância encontrada, basta analisar a faixa de largura 2δ centrada na divisão.

Recorrência: $T(n) = 2T(n/2) + \Theta(n)$, para n maior que o limiar escolhido.
Resultado: $T(n) = \Theta(n \log n)$, incluindo as ordenações iniciais de $\Theta(n \log n)$. A cada nível, repartir por y e analisar a faixa custa $\Theta(n)$.

Por que somente 7 vizinhos por ponto?

Comparamos a partir de cada ponto p apenas os próximos na ordem de y com diferença vertical de até δ. Considere o retângulo de largura 2δ e altura δ acima de p. Ele pode ser separado em duas metades de δ × δ; em cada metade, quatro quadrados de lado δ/2. Dois pontos no mesmo quadrado estariam a uma distância menor que δ, contrariando a menor distância já encontrada dentro da metade correspondente. Assim, pode haver no máximo 8 pontos nessa região, contando p: basta comparar até 7 sucessores. Essa verificação mantém a combinação em tempo linear.

Limiar de implementação

Adotamos limiar 32: subproblemas de até 32 pontos são resolvidos diretamente. Esse valor foi fixado por uma decisão inicial de implementação, reduzindo a quantidade de chamadas recursivas, sem alterar a complexidade assintótica. O estudo posterior de limiares testa se essa escolha foi de fato eficiente (última página e limiar_medicoes.csv).

Exemplo de saída

Para os pontos (0,0), (3,4), (1,1), a resposta é o par de índices (0,2), com distância euclidiana $\sqrt{2} \approx 1{,}4142$. Um dos testes verifica esse resultado em ambas as implementações.

Etapa 3 — Metodologia

Testes, repetição, geração de dados e ambiente

Planejamento do experimento

Para cada um dos cinco algoritmos foram usadas entradas com n = 100, 200, 400, 800, 1.600, 3.200 e 6.400. Cada tamanho teve 5 execuções medidas, além de uma chamada de aquecimento não contabilizada. O relógio foi time.perf_counter_ns(), com conversão de nanossegundos para milissegundos. A tabela apresenta a média das cinco medidas; o CSV bruto preserva cada execução.

Entradas e comparabilidade

O gerador pseudoaleatório utiliza semente fixa 2026. No subarranjo, cada entrada contém inteiros uniformes entre -100 e 100. Nos pontos, x e y são números reais uniformes entre -5.000 e 5.000. Para cada problema e tamanho, os métodos recebem exatamente os mesmos dados. As funções são executadas em um único processo e em sequência.

Configuração da máquina medida

Sistema: Linux-6.18.57-x86_64-with-glibc2.41

CPU reportada: AMD EPYC 9V74 80-Core Processor

CPUs lógicas visíveis: 5

Versão Python: 3.13.5

Momento do experimento (UTC): 2026-11-06T02:44:24+00:00

Validação de correção

O arquivo JSON contém 9 casos previstos de subarranjo e 6 casos previstos de pontos. Inclui vetores de tamanho 1, valores apenas negativos, empates, pares repetidos, coordenadas negativas e um par que cruza a divisão. Os testes automatizados também geram vetores e pontos aleatórios para confrontar força bruta e divisão e conquista em diferentes tamanhos e limiares.

Reprodutibilidade e limites

Os resultados a seguir são medições reais do ambiente que executou o script, não medições feitas no computador pessoal do integrante. Executar python benchmark.py em outro equipamento gera CSVs e gráficos próprios. Ruído do sistema operacional, cache, interpretador e carga da CPU influenciam especialmente os tempos mais curtos.

Exemplos de testes com saída conhecida

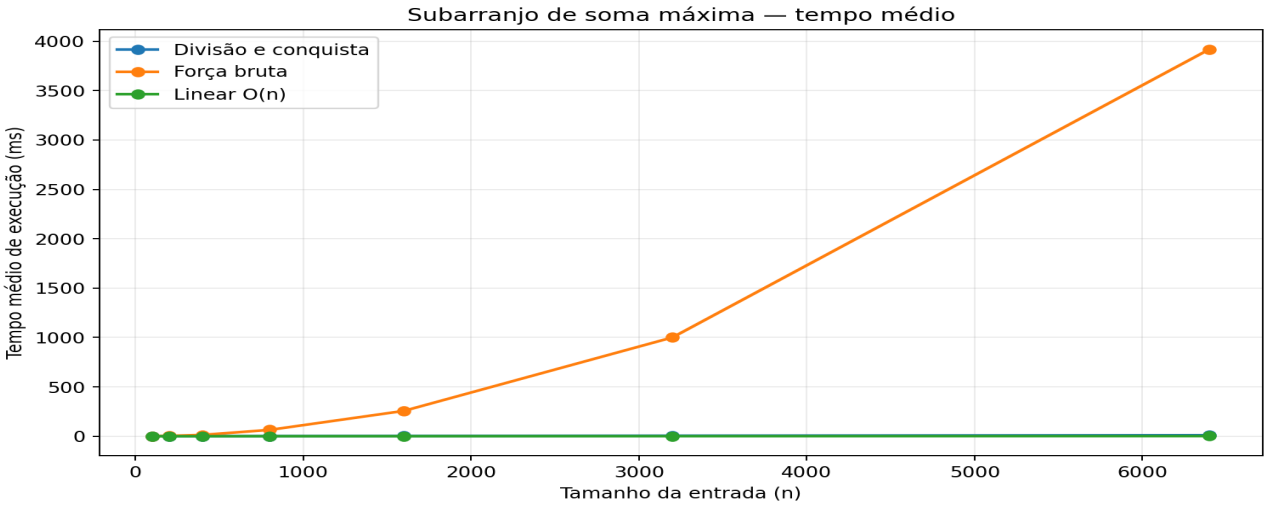
Cenário	Entrada (resumo)	Resultado esperado
Só negativos	[-8,-3,-6,-2,-5,-4]	soma -2; [3,3]
Tamanho 1	[7]	soma 7; [0,0]
Empate	[0,0,0]	soma 0; [0,0]
Ponto repetido	(2,2), (10,10), (2,2)	distância 0
Par único	(0,0), (3,4)	distância 5

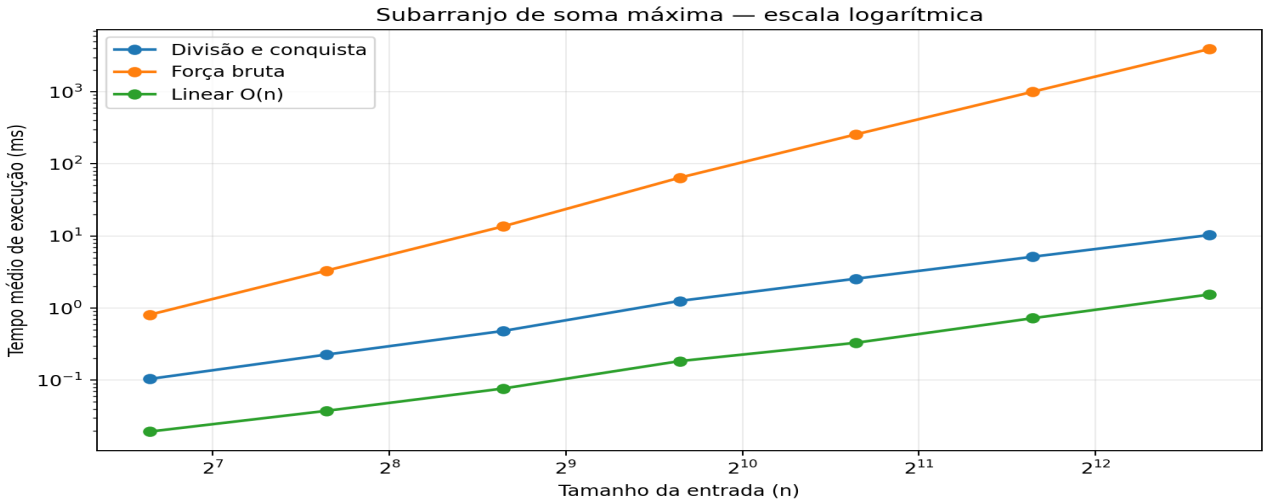
Resultados — Subarranjo máximo

Médias reais em milissegundos, com 5 execuções por configuração

Tabela 1. Tempos médios medidos (ms)

n	Força bruta	Divisão e conquista	Linear O(n)
100	0,808	0,104	0,019
200	3,302	0,226	0,038
400	13,550	0,481	0,076
800	64,215	1,256	0,184
1.600	256,767	2,558	0,330
3.200	1000,720	5,148	0,723
6.400	3919,280	10,309	1,543



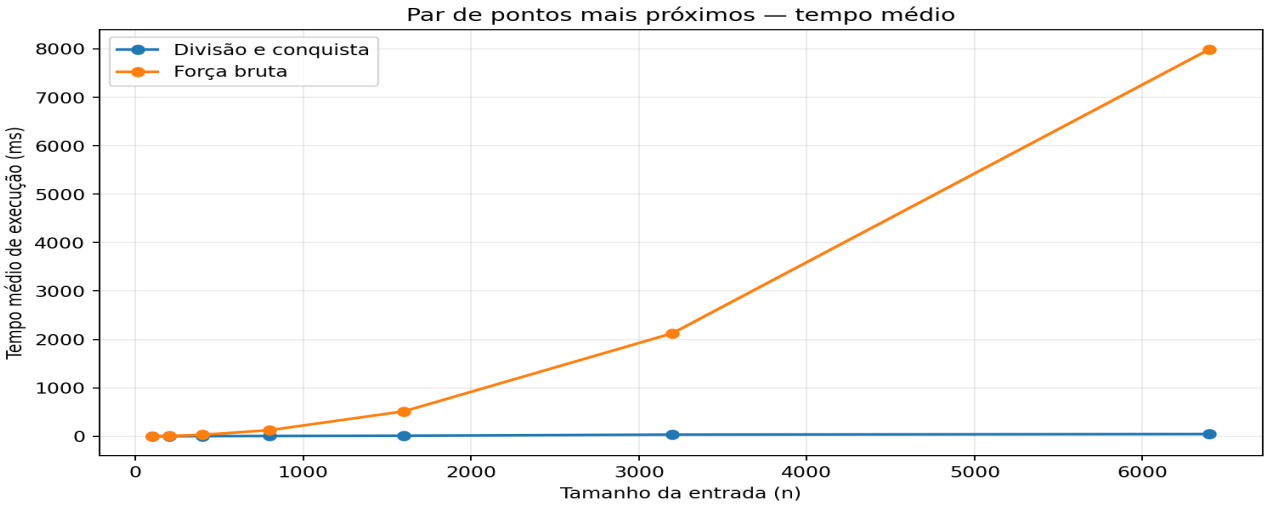


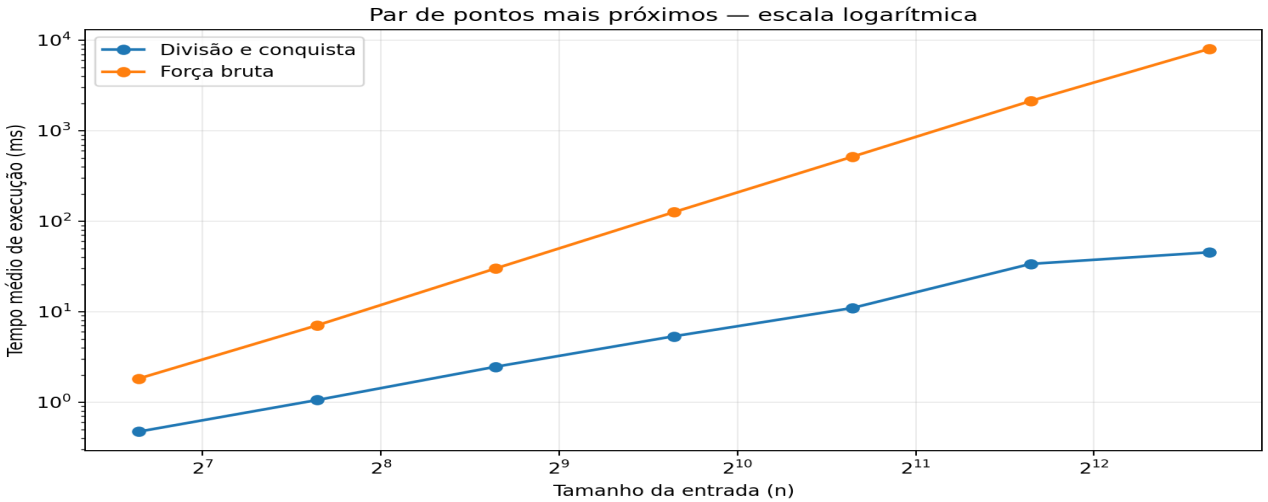
Resultados — Pontos mais próximos

Médias reais em milissegundos, com 5 execuções por configuração

Tabela 2. Tempos médios medidos (ms)

n	Força bruta	Divisão e conquista
100	1,823	0,473
200	7,065	1,056
400	29,983	2,461
800	125,773	5,345
1.600	514,639	10,964
3.200	2126,855	33,691
6.400	7992,562	45,247





Discussão e conclusão

Crescimento dos tempos, limiar e síntese dos resultados

Interpretação dos gráficos

Em n = 6.400, a força bruta do subarranjo levou 3919,280 ms, contra 10,309 ms por divisão e conquista (aprox. 380.2× mais rápida) e 1,543 ms no método linear. Nos pontos, a força bruta levou 7992,562 ms e a divisão e conquista 45,247 ms (aprox. 176.6× mais rápida). Os gráficos logarítmicos tornam visíveis as diferenças de tendência que a escala linear comprime.

Ao duplicar n, o custo quadrático tende a se multiplicar por aproximadamente 4; para Θ(n log n), cresce por pouco mais de 2; para Θ(n), próximo de 2. As medições não reproduzem razões exatas por causa de custos constantes e variações de execução. Em entradas pequenas, a força bruta pode ser competitiva por evitar recursão e ordenações, mas perde eficiência conforme n aumenta.

Estudo do limiar da Etapa 2

n / limiar	2	8	16	32	64
100	0,259	0,219	0,291	0,492	0,903
800	3,172	2,707	3,163	4,662	8,154
3200	32,643	13,274	17,622	22,808	39,924
6400	39,442	30,483	45,411	49,318	79,092

Tabela 3. Média de 5 execuções (ms) por limiar. Nesta máquina e nas entradas testadas, o valor 8 foi mais rápido que 32: nossa escolha inicial de 32 não foi a mais eficiente. O método continuou correto e O(n log n), mas esse parâmetro merece ajuste em uma próxima versão.

Conclusão

Os cinco algoritmos foram implementados e confrontados com testes conhecidos e aleatórios. Os experimentos sustentam as tendências teóricas: Θ(n^2) para força bruta, Θ(n log n) para divisão e conquista e Θ(n) para o desafio de subarranjo. A técnica recursiva torna-se especialmente vantajosa nas entradas grandes. As medições, testes e códigos são reproduzíveis pelos arquivos do projeto.

Observação do enunciado: a expressão "6.400 dígitos" na Etapa 3 foi interpretada como 6.400 elementos/pontos, já que estes são os dados dos dois problemas. O repositório Git deve refletir o desenvolvimento real do autor.

ETAPA 3 • EXPERIMENTO COMPLEMENTAR

Medições no computador do autor

Medições locais: Windows 11, Intel Core i5, Python 3.11.9

Ambiente e protocolo

Sistema: Microsoft Windows 11 Home Single Language.
CPU: Intel Core i5-1235U (12ª geração); RAM visível ao sistema: 7,7 GB.
Python 3.11.9; cinco medições por configuração; tempos em milissegundos.
Entradas: n = 100, 200, 400 e 800; gráficos gerados com as médias abaixo.

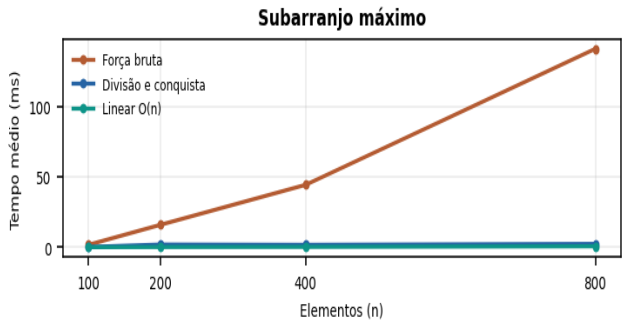
Tabela A — Subarranjo de soma máxima

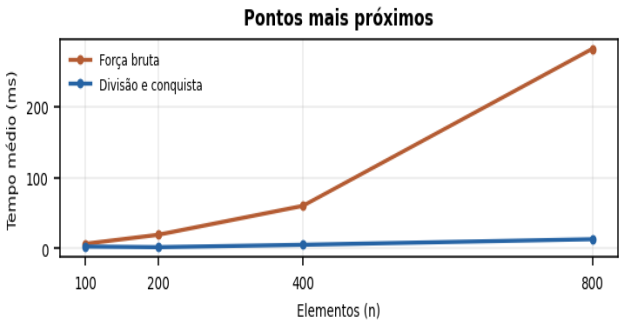
n	Força bruta	Divisão e conquista	Linear O(n)
100	1.7218	0.2130	0.0564
200	15.9920	1.9204	0.1739
400	44.2613	1.7251	0.2734
800	140.7029	2.3731	0.6855

Tabela B — Par de pontos mais próximos

n	Força bruta	Divisão e conquista
100	6.8546	2.9125
200	19.4678	2.0177
400	60.3290	5.4007
800	282.1655	13.2566

Gráficos — comparação das médias





Interpretação

Em n = 800, divisão e conquista foi cerca de 59x mais rápida no subarranjo e 21x no problema de pontos. As medidas variam entre execuções.

As páginas 4–7 registram o experimento de referência feito em outra máquina, com tamanhos até 6.400. Esta página registra separadamente os resultados locais.

A execução completa local e os CSVs devem ser preservados no repositório Git.